

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle TI

Rappels de programmation web : HTML et JavaScript

Module 1 : Programmation

Cette fiche propose des exercices de synthèse mobilisant la structure HTML, la manipulation du DOM et la validation client en JavaScript. Le code doit être conforme aux standards actuels (HTML5, ES6+).

EXERCICE 1 Gestion dynamique d'un catalogue de modules

On souhaite développer une interface d'administration pour un établissement de formation. La page unique `admin.html` doit permettre de saisir des modules (code, intitulé, volume horaire, coefficient), de les afficher dans un tableau récapitulatif triable par colonne, et de calculer le total des volumes horaires et la moyenne des coefficients. Aucune base de données ni backend n'est utilisé ; la persistance se fait via `localStorage`.

1. **Structure HTML (3 pts).** Écrire le squelette HTML5 complet. Le formulaire de saisie (identifié `form-module`) doit contenir : un `select` pour le semestre (S1, S2, S3, S4), des `input` adaptés pour les autres champs (types, contraintes `min`, `max`, `pattern`, `required`), et deux boutons : « Ajouter » (type `submit`) et « Réinitialiser » (type `reset`). Prévoir une section pour le tableau (identifié `vue-tableau`) avec `thead`, `tbody` (identifié `corps-tableau`), `tfoot` pour les totaux, et une zone d'alerte (identifiée `zone-message`) pour les retours utilisateur.
2. **Modèle de données et stockage (3 pts).** Définir la structure d'un objet `Module` en JavaScript. Écrire la fonction `chargerCatalogue()` qui récupère le tableau d'objets depuis `localStorage` (clé `'catalogueModules'`), gère l'absence de donnée (retourne un tableau vide), et appelle `rendreTableau()`.
3. **Rendu et tri (4 pts).** Implémenter `rendreTableau(liste)`. Cette fonction vide `corps-tableau`, génère les `tr/td` pour chaque module, ajoute une colonne d'action avec un bouton « Supprimer » (attribut `data-code` pour identifier le module), met à jour le `tfoot` (total volume horaire, moyenne coefficient arrondie à 2 décimales). Implémenter le tri dynamique : un clic sur un `th` (sauf colonne action) trie `liste` selon la colonne (ordre croissant/décroissant alterné, indicateur visuel ▼/▲ dans le `th`) et rafraîchit l'affichage.
4. **Gestion des événements et validation (4 pts).** Au `submit` du formulaire : empêcher le rechargement, valider côté client (code unique non vide, volume horaire entier > 0, coefficient nombre > 0). En cas d'erreur, afficher le message dans `zone-message` (classe `erreur`) sans vider le formulaire. En cas de succès : ajouter l'objet au catalogue, sauvegarder dans `localStorage`, rafraîchir le tableau, vider le formulaire, afficher un message de succès (classe `succes`). Gérer la suppression via délégation d'événement sur `corps-tableau` (confirmation `confirm()` avant suppression effective).
5. **Analyse critique (2 pts).** Expliquer pourquoi la validation côté client est insuffisante

pour garantir l'intégrité des données dans une application réelle. Citer deux types d'attaques ou de contournements possibles. Proposer une architecture complémentaire.

EXERCICE 2 Formulaire d'inscription multi-étapes avec validation contextuelle

On conçoit un formulaire d'inscription en trois étapes (fieldsets) : 1. Identité (Nom, Prénom, Email, Date de naissance), 2. Coordonnées (Adresse, Ville, Code postal, Téléphone), 3. Sécurité (Mot de passe, Confirmation, Question secrète, Réponse). La navigation se fait par boutons « Suivant » / « Précédent » ; le bouton « Valider » n'apparaît qu'à l'étape 3. Un indicateur de progression (barre ou étapes numérotées) montre l'avancement.

1. Architecture HTML sémantique (3 pts). Produire le code HTML du formulaire (`id="inscription"`). Utiliser `fieldset` pour chaque étape (attribut `hidden` géré en JS), `legend` pour les titres. Associer correctement chaque `label` à son `input` (`for/id`). Choisir les types `d'input` les plus pertinents (`email`, `tel`, `date`, `password`, `number` pour code postal). Ajouter les attributs de contrainte HTML5 (`required`, `minlength`, `pattern` pour le téléphone français, `min/max` pour la date de naissance : majeur, < 100 ans). Prévoir un `output` ou `span` de classe `erreur-champ` après chaque champ pour les messages d'erreur JS.
2. Machine à états de navigation (3 pts). En JavaScript, créer un objet `FormWizard` (ou module IIFE) encapsulant : l'index de l'étape courante, la liste des `fieldset`, les références aux boutons de navigation. Implémenter `afficherEtape(index)` qui gère l'attribut `hidden`, l'état `disabled` des boutons « Précédent » (étape 0) / « Suivant » (dernière étape), et met à jour l'indicateur de progression.
3. Validation différentielle par étape (4 pts). Implémenter `validerEtapeActuelle()`. Cette fonction ne valide que les champs visibles de l'étape courante. Elle utilise l'API `Constraint Validation` (`checkValidity()`, `validity`) pour la validation native, mais ajoute des règles personnalisées : unicité de l'email (simulée via un `Set` d'emails « déjà pris » fourni), cohérence « Mot de passe / Confirmation » (étape 3), force du mot de passe (regex : 8 car., 1 maj, 1 min, 1 chiffre, 1 spécial). Afficher les messages d'erreur dans les `span.erreur-champ` correspondants et ajouter la classe `invalide` sur l'`input`. Retourner `true` si l'étape est valide. Le bouton « Suivant » ne passe à l'étape suivante que si cette fonction retourne `true`.
4. Soumission finale et agrégation (2 pts). Au `submit` final (étape 3 valide), construire un objet `FormData` ou un objet littéral contenant toutes les données des 3 étapes (exclure la confirmation de mot de passe). Simuler l'envoi `fetch('/api/inscription', ...)` avec `JSON.stringify`. Gérer la réponse (succès/erreur réseau) et afficher un message global.
5. Accessibilité et UX (2 pts). Citer 3 attributs ARIA ou rôles HTML à ajouter pour rendre ce

formulaire multi-étapes accessible aux lecteurs d'écran (navigation, région live pour erreurs, indication d'étape courante). Justifier l'intérêt de la validation au fur et à mesure (sur événement `input` ou `blur`) en plus de la validation à la navigation.

EXERCICE 3 Tableau de bord interactif : filtrage, pagination et export

Une page `dashboard.html` affiche une liste d'étudiants (Matricule, Nom, Prénom, Filière, Niveau, Moyenne) dans un tableau paginé (10 lignes/page). Les données initiales (50 entrées) sont fournies dans un fichier `data/etudiants.json` chargé via `fetch` au chargement de la page. L'interface propose : un champ de recherche global (filtre sur toutes les colonnes textuelles), un sélecteur de filtre par Filière, un tri par colonne (clic sur en-tête), et un bouton « Exporter CSV » (uniquement les lignes filtrées et triées courantes).

1. Chargement asynchrone et état initial (2 pts). Écrire la fonction asynchrone `initDashboard()`. Elle doit : afficher un état « Chargement... » dans `tbody`, récupérer le JSON via `fetch`, gérer les erreurs HTTP/réseau (affichage message d'erreur dans le tableau), stocker les données brutes dans une variable module `let donneesBrutes = []`, et appeler `appliquerFiltresEtRendu()`.
2. Pipeline de transformation des données (4 pts). Implémenter `appliquerFiltresEtRendu()`. Cette fonction chaîne les opérations pures (sans effet de bord sur le DOM) sur `donneesBrutes` :
 - (a) Filtre texte (insensible à la casse, recherche partielle sur Nom, Prénom, Matricule).
 - (b) Filtre Filière (valeur « Toutes » = pas de filtre).
 - (c) Tri (selon `colonneTri` et `ordreTri` stockés en état).

Elle met à jour `donneesFiltrees` (résultat final), réinitialise la page courante à 1, et appelle `rendrePage()` et `rendrePagination()`.

3. Rendu paginé et performances (3 pts). Implémenter `rendrePage()`. Calculer les indices de début/fin selon `pageCourante` et `TAILLE_PAGE=10`. Utiliser un `DocumentFragment` pour construire les `tr` (création via `document.createElement` ou `template content.cloneNode(true)`) et les ajouter au fragment, puis vider `tbody` (`innerHTML=''`) et y ajouter le fragment en une seule opération. Mettre à jour l'affichage « Page X sur Y / N résultats totaux ».
4. Génération CSV côté client (3 pts). Implémenter `exporterCSV()`. Construire la chaîne CSV (en-têtes + lignes de `donneesFiltrees`) en échappant correctement les guillemets doubles et les retours à la ligne dans les cellules (RFC 4180). Créer un `Blob` de type `text/csv; charset=utf-8;`, générer une URL objet (`URL.createObjectURL`), déclencher le téléchargement via un lien `<a download="etudiants_export.csv">` simulé, puis révoquer l'URL. Gérer le BOM UTF-8 (`\uFEFF`) pour Excel.
5. Architecture et maintenabilité (2 pts). Le code actuel mélange logique métier (filtres,

tri, CSV) et manipulation DOM dans les mêmes fonctions. Proposer une refactorisation selon le pattern MVC ou Modèle-Vue-Contrôleur (ou MVVM) adaptée au Vanilla JS. Décrire succinctement les responsabilités de chaque couche (Modèle : état + données + persistance ; Vue : rendu + écouteurs UI ; Contrôleur : orchestration). Quel avantage pour les tests unitaires ?

EXERCICE 4 Sécurité et limites de la validation côté client

On considère l'exercice 1 (Catalogue de modules). Un étudiant malveillant ouvre les outils de développement (F12), modifie le HTML pour supprimer l'attribut `required` sur le champ « Code », change le `type="number"` du volume horaire en `type="text"`, et injecte un `<script>alert(1)</script>` dans l'intitulé via la console : `document.querySelector('#intitule').value = '<script>alert(1)</script>'`.

1. Contournement des contraintes HTML5 (2 pts). Expliquer pourquoi la suppression des attributs `required`, `min`, `max`, `pattern` dans l'inspecteur permet de soumettre des données invalides malgré le code JavaScript de validation écrit dans l'exercice 1 (question 4). Quel événement JavaScript est déclenché par le `submit` natif du formulaire même si les contraintes HTML sont violées après modification du DOM ?
2. Injection XSS stockée via `localStorage` (3 pts). Si la fonction `rendreTableau()` insère l'intitulé via `cellule.textContent = module.intitule`, l'injection est-elle neutralisée ? Et si elle utilise `cellule.innerHTML += module.intitule` ? Démontrer par l'exemple. Comment l'attaque devient-elle « stockée » et persistante pour les autres utilisateurs (si partie partagée) ou pour la même session ? Proposer la fonction d'échappement HTML minimale à appliquer systématiquement avant tout `innerHTML`.
3. Validation du code unique : faille de course (TOCTOU) (2 pts). La vérification « code unique » se fait en JS en parcourant `catalogueModules` avant `push`. Imaginer un scénario (même utilisateur, onglets multiples, ou manipulation console rapide) où deux modules avec le même code sont ajoutés. Expliquer le principe Time-Of-Check to Time-Of-Use. Pourquoi `localStorage` n'offre-t-il pas de transactions atomiques ?
4. Principes de défense en profondeur (3 pts). Citer et expliquer 3 mesures complémentaires côté serveur (backend) indispensables pour sécuriser réellement cette fonctionnalité d'ajout de module, même si le frontend est parfaitement codé. (Indices : validation schéma, authentification/autorisation, en-têtes HTTP, sanitization sortie).
5. Content Security Policy (2 pts). Écrire l'en-tête HTTP `Content-Security-Policy` le plus restrictif possible pour cette page (scripts inline interdits, styles inline interdits, fontes/images self, pas de frames, pas d'objets), qui aurait empêché l'exécution du `<script>alert(1)</script>` même s'il avait été injecté via `innerHTML`.